modeling-core-mahin

Mahin Roddur

2026-04-08

```r
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/roddu/Downloads/glassdoor-company-reviews
```

```r
data <- read.csv(here("data", "processed", "reviews_with_sent_scores.csv"))
```

```r
data <- data %>%
  mutate(employee_length_bin = case_when(
    employee_length <= 1 ~ "short",
    employee_length < 4 ~ "medium",
    TRUE ~ "long"
  )) %>%
  mutate(employee_length_bin = factor(
    employee_length_bin,
    levels = c("short", "medium", "long"),
    ordered = TRUE
  ))

features_base <- c(
  # Ratings
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion",

  # Context
  "employee_job_quit",
  "covid_related_review",
  "rating_year",

  # Missingness
  "career_missing",

  # Engagement
  "helpfulness_score",
```

```
  # Text structure
  "summary_wc",
  "pros_wc",
  "cons_wc",
  "advice_wc"
)
```

Scaling helper

```
scale_train_test <- function(train_df, test_df, feature_cols) {
  train_scaled <- train_df
  test_scaled  <- test_df

  train_means <- sapply(train_df[feature_cols], mean, na.rm = TRUE)
  train_sds   <- sapply(train_df[feature_cols], sd, na.rm = TRUE)

  train_sds[train_sds == 0 | is.na(train_sds)] <- 1

  train_scaled[feature_cols] <- scale(train_df[feature_cols],
                                      center = train_means,
                                      scale = train_sds)

  test_scaled[feature_cols] <- scale(test_df[feature_cols],
                                     center = train_means,
                                     scale = train_sds)

  list(
    train = train_scaled,
    test = test_scaled,
    center = train_means,
    scale = train_sds
  )
}
```

Train

```
split_data <- function(df, target, p = 0.8, seed = 123) {
  set.seed(seed)
  idx <- createDataPartition(df[[target]], p = p, list = FALSE)
  list(
    train = df[idx, ],
    test  = df[-idx, ]
  )
}

rmse_vec <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}

prepare_model_data <- function(df, target, feature_cols) {
```

```r
  cols_needed <- c(target, feature_cols)

  df_model <- df %>%
    dplyr::select(all_of(cols_needed)) %>%
    filter(!is.na(.data[[target]]))

  df_model
}
```

```r
run_tenure_lm <- function(df, feature_cols, target = "employee_length", seed = 123) {
  df_model <- prepare_model_data(df, target, feature_cols)

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  formula_lm <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- lm(formula_lm, data = train_scaled)

  preds <- predict(model, newdata = test_scaled)
  rmse  <- rmse_vec(test_scaled[[target]], preds)

  coef_tbl <- broom::tidy(model) %>%
    filter(term != "(Intercept)") %>%
    arrange(desc(abs(estimate)))

  list(
    model = model,
    rmse = rmse,
    predictions = preds,
    actual = test_scaled[[target]],
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

tenure_lm_results <- run_tenure_lm(
  df = data,
  feature_cols = features_base,
  target = "employee_length"
)

tenure_lm_results$rmse
```

```
## [1] 4.455652
```

```r
tenure_lm_results$coefficients
```

```
## # A tibble: 15 x 5
##    term                          estimate std.error statistic  p.value
##    <chr>                            <dbl>     <dbl>     <dbl>    <dbl>
##  1 rating_senior_leadership     -0.919       0.262   -3.51   0.000481
##  2 rating_culture_values         0.504       0.267    1.89   0.0597
##  3 rating_year                  -0.472       0.156   -3.03   0.00251
##  4 career_missing               -0.423       0.193   -2.19   0.0289
##  5 rating_compensation_benefits  0.292       0.196    1.49   0.137
##  6 cons_wc                      -0.279       0.167   -1.67   0.0952
##  7 rating_career_opportunities   0.258       0.170    1.52   0.129
##  8 helpfulness_score            -0.194       0.162   -1.20   0.231
##  9 covid_related_review          0.150       0.152    0.989  0.323
## 10 employee_job_quit             0.141       0.155    0.907  0.365
## 11 pros_wc                       0.127       0.160    0.794  0.427
## 12 advice_wc                     0.111       0.182    0.608  0.543
## 13 summary_wc                    0.0412      0.152    0.271  0.786
## 14 rating_work_life             -0.0271      0.201   -0.135  0.893
## 15 rating_diversity_inclusion    0.00513     0.210    0.0244 0.981
```

```r
summary(tenure_lm_results$model)
```

```
##
## Call:
## lm(formula = formula_lm, data = train_scaled)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.0599 -2.2492 -1.1693  0.7837 17.7216
##
## Coefficients:
##                               Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   3.834165   0.143648  26.691  < 2e-16 ***
## rating_culture_values         0.503975   0.267252   1.886 0.059694 .
## rating_work_life             -0.027141   0.200966  -0.135 0.892605
## rating_career_opportunities   0.258492   0.170194   1.519 0.129212
## rating_compensation_benefits  0.291688   0.195716   1.490 0.136529
## rating_senior_leadership     -0.919206   0.262181  -3.506 0.000481 ***
## rating_diversity_inclusion    0.005129   0.210404   0.024 0.980558
## employee_job_quit             0.140943   0.155355   0.907 0.364562
## covid_related_review          0.150381   0.152113   0.989 0.323157
## rating_year                  -0.471937   0.155686  -3.031 0.002515 **
## career_missing               -0.422518   0.193072  -2.188 0.028934 *
## helpfulness_score            -0.193745   0.161560  -1.199 0.230807
## summary_wc                    0.041250   0.152103   0.271 0.786310
## pros_wc                       0.126718   0.159610   0.794 0.427479
## cons_wc                      -0.278869   0.166915  -1.671 0.095175 .
## advice_wc                     0.110776   0.182058   0.608 0.543055
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 4.068 on 786 degrees of freedom
## Multiple R-squared:  0.05694,    Adjusted R-squared:  0.03894
## F-statistic: 3.164 on 15 and 786 DF,  p-value: 4.498e-05
```

```r
run_tenure_ordinal <- function(df, feature_cols, target = "length_bin", seed = 123) {
  df_model <- prepare_model_data(df, target, feature_cols)

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  formula_ord <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- polr(formula_ord, data = train_scaled, Hess = TRUE)

  prob_preds <- predict(model, newdata = test_scaled, type = "probs")
  class_preds <- predict(model, newdata = test_scaled, type = "class")

  acc <- mean(as.character(class_preds) == as.character(test_scaled[[target]]), na.rm = TRUE)

  coef_tbl <- broom::tidy(model) %>%
    filter(!grepl("\\|", term)) %>%
    arrange(desc(abs(estimate)))

  list(
    model = model,
    accuracy = acc,
    class_predictions = class_preds,
    actual = test_scaled[[target]],
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

tenure_ord_results <- run_tenure_ordinal(
  df = data,
  feature_cols = features_base,
  target = "employee_length_bin"
)

tenure_ord_results$accuracy
```

```
## [1] 0.4070352
```

```r
tenure_ord_results$coefficients
```

```
## # A tibble: 15 x 5
##    term                          estimate std.error statistic coef.type
##    <chr>                            <dbl>     <dbl>     <dbl> <chr>
##  1 rating_senior_leadership      -0.358     0.121     -2.96  coefficient
##  2 rating_culture_values          0.228     0.123      1.85  coefficient
##  3 rating_compensation_benefits   0.125     0.0908     1.38  coefficient
##  4 career_missing                -0.122     0.0912    -1.34  coefficient
##  5 rating_diversity_inclusion     0.122     0.0952     1.28  coefficient
##  6 rating_year                   -0.113     0.0739    -1.53  coefficient
##  7 employee_job_quit             -0.0826    0.0703    -1.17  coefficient
##  8 pros_wc                        0.0704    0.0779     0.904 coefficient
##  9 rating_career_opportunities   -0.0676    0.0798    -0.847 coefficient
## 10 cons_wc                       -0.0626    0.0803    -0.779 coefficient
## 11 summary_wc                    -0.0421    0.0692    -0.609 coefficient
## 12 advice_wc                     -0.0379    0.0872    -0.435 coefficient
## 13 rating_work_life              -0.0361    0.0915    -0.395 coefficient
## 14 helpfulness_score              0.0301    0.0738     0.408 coefficient
## 15 covid_related_review          -0.0107    0.0692    -0.155 coefficient
```

```r
summary(tenure_ord_results$model)
```

```
## Call:
## polr(formula = formula_ord, data = train_scaled, Hess = TRUE)
##
## Coefficients:
##                                 Value Std. Error t value
## rating_culture_values         0.22758    0.12315  1.8479
## rating_work_life             -0.03610    0.09146 -0.3947
## rating_career_opportunities  -0.06761    0.07982 -0.8470
## rating_compensation_benefits  0.12509    0.09080  1.3777
## rating_senior_leadership     -0.35782    0.12087 -2.9605
## rating_diversity_inclusion    0.12181    0.09518  1.2798
## employee_job_quit            -0.08255    0.07031 -1.1740
## covid_related_review         -0.01072    0.06922 -0.1548
## rating_year                  -0.11310    0.07393 -1.5299
## career_missing               -0.12214    0.09119 -1.3394
## helpfulness_score             0.03014    0.07385  0.4082
## summary_wc                   -0.04214    0.06922 -0.6089
## pros_wc                       0.07043    0.07795  0.9036
## cons_wc                      -0.06258    0.08031 -0.7793
## advice_wc                    -0.03793    0.08718 -0.4351
##
## Intercepts:
##              Value    Std. Error t value
## short|medium  -1.1132   0.0825   -13.4971
## medium|long    0.3091   0.0724     4.2691
##
## Residual Deviance: 1697.466
## AIC: 1731.466
```

Logistic Regression

```r
run_recommend_logit <- function(df, feature_cols, target = "flags_recommend_frend", seed = 123) {
  df_model <- prepare_model_data(df, target, feature_cols)

  df_model[[target]] <- factor(df_model[[target]], levels = c("NEGATIVE", "POSITIVE"))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  train_scaled[[target]] <- factor(train_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))
  test_scaled[[target]]  <- factor(test_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))

  formula_logit <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- glm(formula_logit, data = train_scaled, family = binomial)

  prob_preds <- predict(model, newdata = test_scaled, type = "response")

  class_preds <- ifelse(prob_preds >= 0.5, "POSITIVE", "NEGATIVE")
  class_preds <- factor(class_preds, levels = levels(test_scaled[[target]]))

  acc <- mean(class_preds == test_scaled[[target]], na.rm = TRUE)

  roc_obj <- pROC::roc(
    response = test_scaled[[target]],
    predictor = prob_preds,
    levels = c("NEGATIVE", "POSITIVE"),
    direction = "<"
  )
  auc_val <- as.numeric(pROC::auc(roc_obj))

  coef_tbl <- broom::tidy(model) %>%
    dplyr::filter(term != "(Intercept)") %>%
    dplyr::mutate(odds_ratio = exp(estimate)) %>%
    dplyr::arrange(desc(abs(estimate)))

  list(
    model = model,
    auc = auc_val,
    accuracy = acc,
    roc = roc_obj,
    probabilities = prob_preds,
    predicted_class = class_preds,
    actual = test_scaled[[target]],
    coefficients = coef_tbl,
```

```r
    train = train_scaled,
    test = test_scaled
  )
}

recommend_results <- run_recommend_logit(
  df = data,
  feature_cols = features_base,
  target = "flags_recommend_frend"
)

recommend_results$auc
```

```
## [1] 0.9117368
```

```r
recommend_results$accuracy
```

```
## [1] 0.8643216
```

```r
recommend_results$coefficients
```

```
## # A tibble: 15 x 6
##    term                          estimate std.error statistic  p.value odds_ratio
##    <chr>                            <dbl>     <dbl>     <dbl>    <dbl>      <dbl>
##  1 helpfulness_score            -1.34e+0     0.403 -3.33     8.82 e-4      0.262
##  2 rating_culture_values         9.34e-1     0.223  4.19     2.80 e-5      2.54
##  3 rating_senior_leadership      8.36e-1     0.200  4.19     2.78 e-5      2.31
##  4 cons_wc                      -6.32e-1     0.290 -2.18     2.91 e-2      0.532
##  5 rating_career_opportunities   5.96e-1     0.176  3.39     6.97 e-4      1.82
##  6 pros_wc                       5.40e-1     0.158  3.43     6.12 e-4      1.72
##  7 rating_work_life              5.28e-1     0.174  3.04     2.40 e-3      1.70
##  8 career_missing               -2.99e-1     0.178 -1.68     9.22 e-2      0.742
##  9 employee_job_quit            -2.93e-1     0.142 -2.07     3.86 e-2      0.746
## 10 rating_compensation_benefits  2.86e-1     0.184  1.56     1.20 e-1      1.33
## 11 advice_wc                     6.79e-2     0.185  0.368    7.13 e-1      1.07
## 12 covid_related_review         -5.00e-2     0.151 -0.332    7.40 e-1      0.951
## 13 rating_year                  -2.57e-2     0.133 -0.194    8.46 e-1      0.975
## 14 summary_wc                   -1.19e-2     0.158 -0.0752   9.40 e-1      0.988
## 15 rating_diversity_inclusion   -6.19e-5     0.199 -0.000312 1.000e+0      1.000
```

```r
summary(recommend_results$model)
```

```
##
## Call:
## glm(formula = formula_logit, family = binomial, data = train_scaled)
##
## Coefficients:
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -2.0353315  0.2208382  -9.216  < 2e-16 ***
## rating_culture_values         0.9335599  0.2228546   4.189 2.80e-05 ***
## rating_work_life              0.5280146  0.1739360   3.036 0.002400 **
```

```
## rating_career_opportunities   0.5961229  0.1758109   3.391 0.000697 ***
## rating_compensation_benefits  0.2858576  0.1837258   1.556 0.119734
## rating_senior_leadership      0.8361815  0.1995447   4.190 2.78e-05 ***
## rating_diversity_inclusion   -0.0000619  0.1987171   0.000 0.999751
## employee_job_quit            -0.2932859  0.1417906  -2.068 0.038598 *
## covid_related_review         -0.0499544  0.1506081  -0.332 0.740127
## rating_year                  -0.0257478  0.1328943  -0.194 0.846375
## career_missing               -0.2990364  0.1776046  -1.684 0.092236 .
## helpfulness_score            -1.3405740  0.4030962  -3.326 0.000882 ***
## summary_wc                   -0.0118824  0.1580748  -0.075 0.940080
## pros_wc                       0.5397319  0.1575197   3.426 0.000612 ***
## cons_wc                      -0.6320353  0.2896538  -2.182 0.029107 *
## advice_wc                     0.0679477  0.1846683   0.368 0.712915
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 989.69  on 800  degrees of freedom
## Residual deviance: 369.67  on 785  degrees of freedom
## AIC: 401.67
##
## Number of Fisher Scoring iterations: 7
```

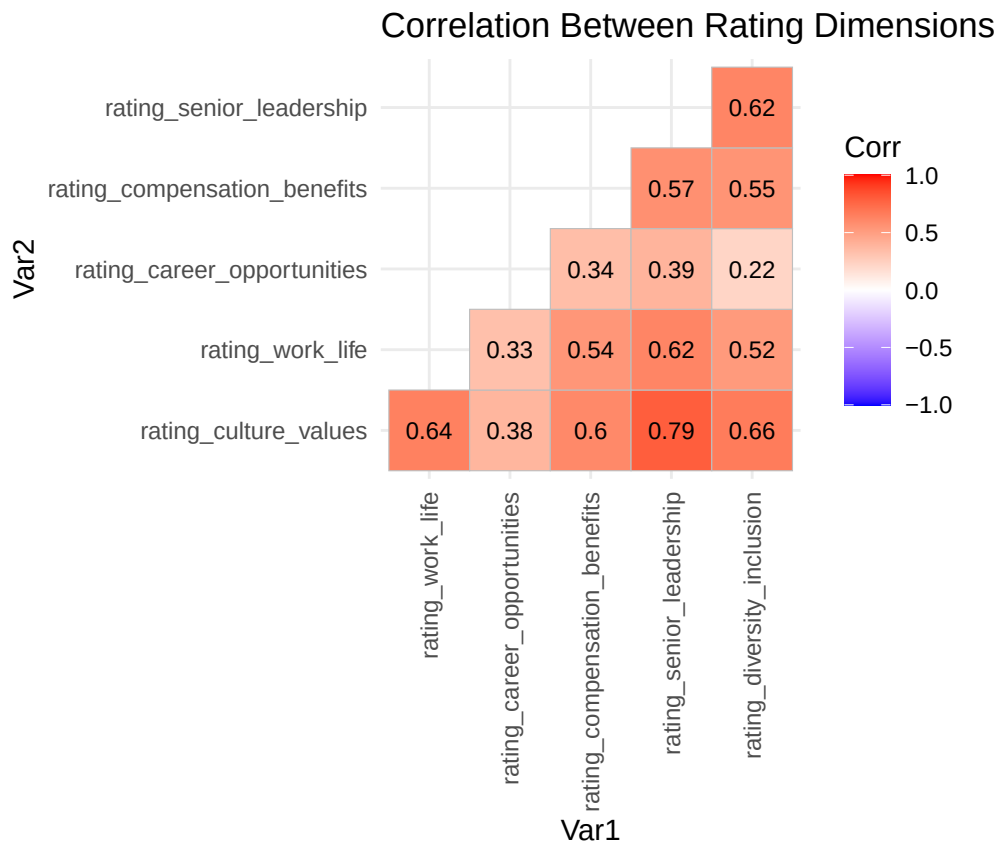
```r
rating_features <- c(
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion"
)

cor_mat <- cor(data[rating_features], use = "pairwise.complete.obs")
library(ggcorrplot)
```

```
## Warning: package 'ggcorrplot' was built under R version 4.5.3
```

```r
cor_mat <- cor(data[rating_features], use = "pairwise.complete.obs")

ggcorrplot(
  cor_mat,
  type = "lower",
  lab = TRUE,
  lab_size = 3,
  colors = c("blue", "white", "red"),
  title = "Correlation Between Rating Dimensions"
) +
  theme_minimal() + theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
```

```
## i The deprecated feature was likely used in the ggcorrplot package.
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once per session.
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```




```r
library(car)
```

```
## Loading required package: carData

##
## Attaching package: ’car’

## The following object is masked from ’package:dplyr’:
##
##     recode

## The following object is masked from ’package:purrr’:
##
##     some
```

```r
check_vif <- function(model) {
  car::vif(model)
}
```

```r
check_vif(tenure_lm_results$model)
```

```
##        rating_culture_values             rating_work_life
##                     3.456992                     1.954810
##  rating_career_opportunities rating_compensation_benefits
##                     1.401990                     1.854002
##     rating_senior_leadership   rating_diversity_inclusion
##                     3.327067                     2.142715
##            employee_job_quit         covid_related_review
##                     1.168170                     1.119932
##                  rating_year               career_missing
##                     1.173160                     1.804246
##            helpfulness_score                   summary_wc
##                     1.263353                     1.119776
##                      pros_wc                      cons_wc
##                     1.233039                     1.348498
##                    advice_wc
##                     1.604271
```

```r
check_vif(recommend_results$model)
```

```
##        rating_culture_values             rating_work_life
##                     1.654179                     1.184187
##  rating_career_opportunities rating_compensation_benefits
##                     1.102448                     1.226940
##     rating_senior_leadership   rating_diversity_inclusion
##                     1.514670                     1.424503
##            employee_job_quit         covid_related_review
##                     1.240966                     1.180091
##                  rating_year               career_missing
##                     1.199966                     1.804231
##            helpfulness_score                   summary_wc
##                     1.249256                     1.180917
##                      pros_wc                      cons_wc
##                     1.475018                     1.955152
##                    advice_wc
##                     2.071738
```

Regularization LASSO and Ridge

```r
run_glmnet_logit <- function(df, feature_cols, target = "flags_recommend_frend", alpha = 1, seed = 123)
  df_model <- prepare_model_data(df, target, feature_cols)

  x <- model.matrix(
    as.formula(paste(target, "~", paste(feature_cols, collapse = " + "))),
    data = df_model
  )[ , -1]

  y <- ifelse(df_model[[target]] == "POSITIVE", 1, 0)
```

```r
  set.seed(seed)
  cv_fit <- cv.glmnet(x, y, family = "binomial", alpha = alpha)

  list(
    cv_model = cv_fit,
    lambda_min = cv_fit$lambda.min,
    lambda_1se = cv_fit$lambda.1se,
    coefficients = coef(cv_fit, s = "lambda.min")
  )
}

# LASSO
run_glmnet_logit(data, features_base, alpha = 1)
```

```
## $cv_model
##
## Call:  cv.glmnet(x = x, y = y, family = "binomial", alpha = alpha)
##
## Measure: Binomial Deviance
##
##       Lambda Index Measure      SE Nonzero
## min 0.007366    42  0.5307 0.03303       9
## 1se 0.022493    30  0.5594 0.02799       8
##
## $lambda_min
## [1] 0.007365519
##
## $lambda_1se
## [1] 0.02249323
##
## $coefficients
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                                lambda.min
## (Intercept)                   -5.91529049
## rating_culture_values          0.66618691
## rating_work_life               0.23579817
## rating_career_opportunities    0.41678065
## rating_compensation_benefits   0.12725133
## rating_senior_leadership       0.57120225
## rating_diversity_inclusion     .
## employee_job_quitTRUE         -0.00410262
## covid_related_review           .
## rating_year                    .
## career_missingTRUE            -0.68234161
## helpfulness_score             -0.21169218
## summary_wc                     .
## pros_wc                        0.01310301
## cons_wc                        .
## advice_wc                      .
```

```r
# Ridge
run_glmnet_logit(data, features_base, alpha = 0)
```

```
## $cv_model
##
## Call:  cv.glmnet(x = x, y = y, family = "binomial", alpha = alpha)
##
## Measure: Binomial Deviance
##
##      Lambda Index Measure      SE Nonzero
## min 0.03340   100  0.5425 0.03342      15
## 1se 0.09294    89  0.5752 0.02837      15
##
## $lambda_min
## [1] 0.03340173
##
## $lambda_1se
## [1] 0.0929423
##
## $coefficients
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                                lambda.min
## (Intercept)                  10.845500301
## rating_culture_values         0.483274800
## rating_work_life              0.247396816
## rating_career_opportunities   0.367821392
## rating_compensation_benefits  0.194999641
## rating_senior_leadership      0.478339926
## rating_diversity_inclusion    0.051288243
## employee_job_quitTRUE        -0.269924621
## covid_related_review         -0.108901742
## rating_year                  -0.007911760
## career_missingTRUE           -0.710295026
## helpfulness_score            -0.112601729
## summary_wc                   -0.001966999
## pros_wc                       0.015089052
## cons_wc                      -0.003629439
## advice_wc                    -0.006120178
```

```r
# Elastic Net
run_glmnet_logit(data, features_base, alpha = 0.5)
```

```
## $cv_model
##
## Call:  cv.glmnet(x = x, y = y, family = "binomial", alpha = alpha)
##
## Measure: Binomial Deviance
##
##      Lambda Index Measure      SE Nonzero
## min 0.00925    47  0.5320 0.03504      11
## 1se 0.04099    31  0.5646 0.02666       8
##
## $lambda_min
## [1] 0.009251521
##
## $lambda_1se
## [1] 0.04098999
```

```
##
## $coefficients
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                                lambda.min
## (Intercept)                  -5.777090041
## rating_culture_values         0.631009447
## rating_work_life              0.245595316
## rating_career_opportunities   0.423477930
## rating_compensation_benefits  0.158365191
## rating_senior_leadership      0.555779636
## rating_diversity_inclusion    .
## employee_job_quitTRUE        -0.118813560
## covid_related_review          .
## rating_year                   .
## career_missingTRUE           -0.759726161
## helpfulness_score            -0.189753526
## summary_wc                    .
## pros_wc                       0.015896371
## cons_wc                      -0.001668672
## advice_wc                    -0.003613765
```

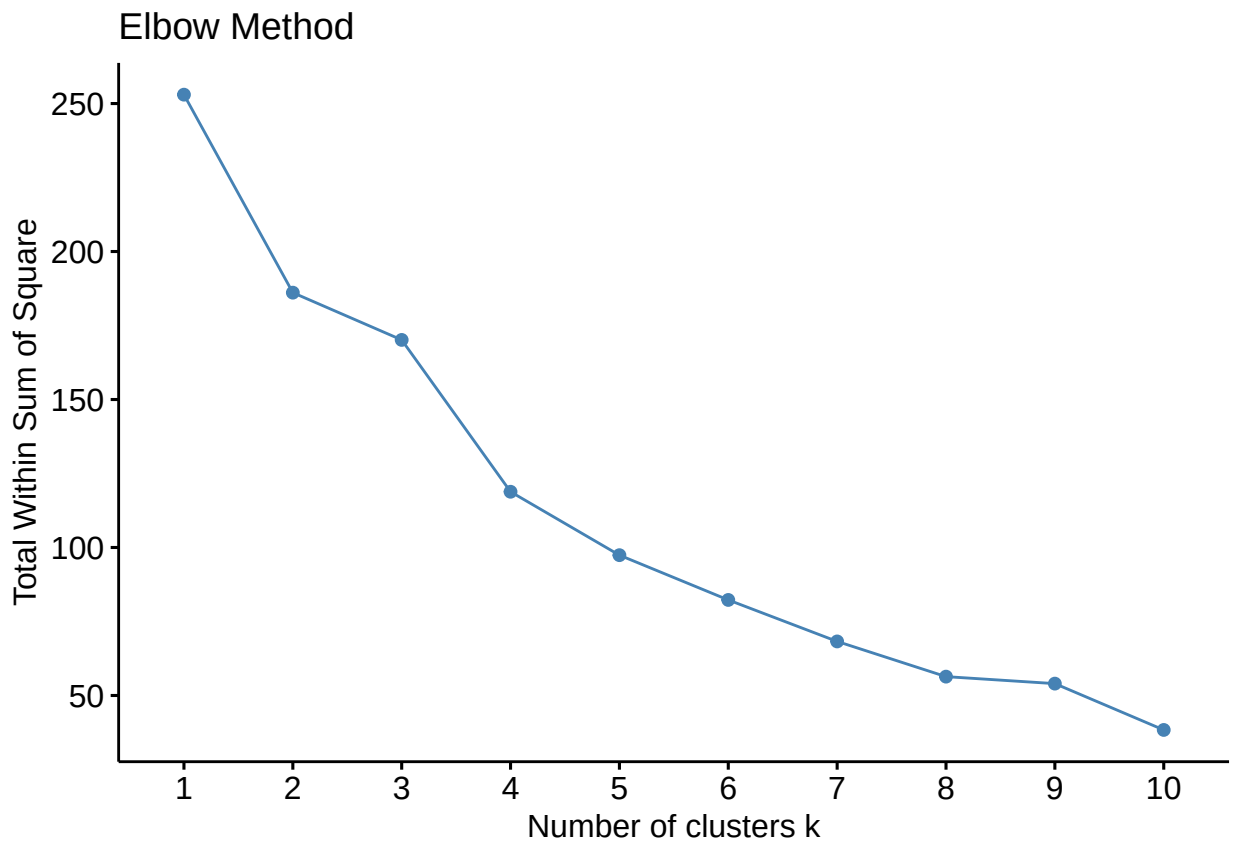
Clustering Prep

```r
aggregate_company_features <- function(df) {
  df %>%
    group_by(company_name, sector) %>%
    summarise(
      n_reviews = n(),

      rating_culture_values = mean(rating_culture_values, na.rm = TRUE),
      rating_work_life = mean(rating_work_life, na.rm = TRUE),
      rating_career_opportunities = mean(rating_career_opportunities, na.rm = TRUE),
      rating_compensation_benefits = mean(rating_compensation_benefits, na.rm = TRUE),
      rating_senior_leadership = mean(rating_senior_leadership, na.rm = TRUE),
      rating_diversity_inclusion = mean(rating_diversity_inclusion, na.rm = TRUE),

      helpfulness_score = mean(helpfulness_score, na.rm = TRUE),
      summary_wc = mean(summary_wc, na.rm = TRUE),
      pros_wc = mean(pros_wc, na.rm = TRUE),
      cons_wc = mean(cons_wc, na.rm = TRUE),
      advice_wc = mean(advice_wc, na.rm = TRUE),

      recommend_rate = mean(flags_recommend_frend == "POSITIVE", na.rm = TRUE),
      quit_rate = mean(employee_job_quit, na.rm = TRUE),

      .groups = "drop"
    )
}

# Remove companies with low reviews
company_df <- aggregate_company_features(data) %>%
  filter(n_reviews >= 5)
```

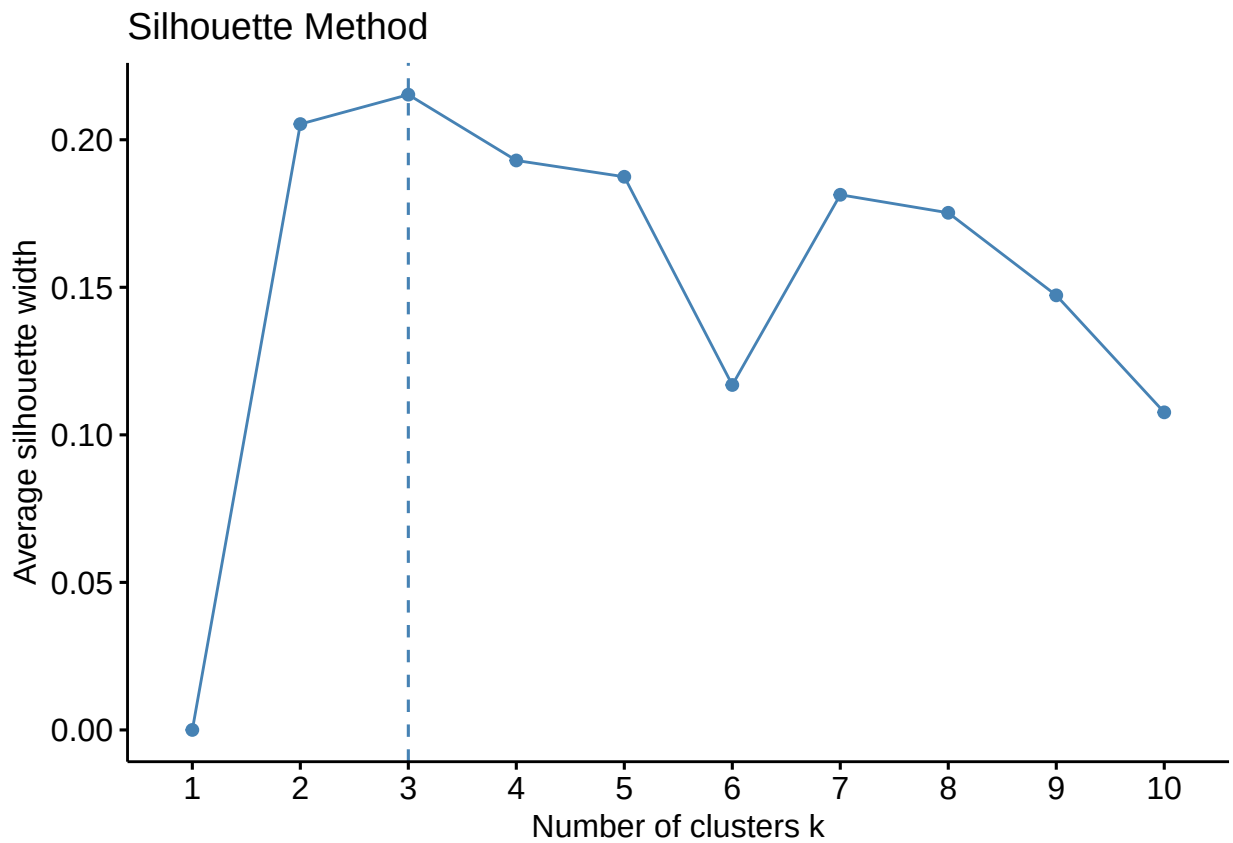
```r
prepare_company_clustering_data <- function(company_df, cluster_features) {
  df_clust <- company_df %>%
    dplyr::select(company_name, sector, n_reviews, all_of(cluster_features))


  scaled_mat <- scale(df_clust[cluster_features])

  list(
    data = df_clust,
    scaled_matrix = scaled_mat
  )
}

cluster_features <- c(
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion",
  "helpfulness_score",
  "summary_wc",
  "pros_wc",
  "cons_wc",
  "advice_wc"
)
```

Choosing clusters

```r
choose_k_elbow <- function(scaled_mat, k_max = 10) {
  fviz_nbclust(scaled_mat, kmeans, method = "wss") +
    ggtitle("Elbow Method")
}

choose_k_silhouette <- function(scaled_mat, k_max = 10) {
  fviz_nbclust(scaled_mat, kmeans, method = "silhouette") +
    ggtitle("Silhouette Method")
}

company_prep <- prepare_company_clustering_data(company_df, cluster_features)

choose_k_elbow(company_prep$scaled_matrix, k_max = 10)
```




```
choose_k_silhouette(company_prep$scaled_matrix, k_max = 10)
```



K-means

```
run_kmeans_clustering <- function(company_df, cluster_features, k = 4, nstart = 25) {
  prep <- prepare_company_clustering_data(company_df, cluster_features)

  set.seed(123)
  km <- kmeans(prep$scaled_matrix, centers = k, nstart = nstart)

  df_out <- prep$data %>%
    mutate(cluster = factor(km$cluster))

  cluster_summary <- df_out %>%
    group_by(cluster) %>%
    summarise(
      n_companies = n(),
      across(all_of(cluster_features), mean, na.rm = TRUE),
      .groups = "drop"
    )

  list(
    model = km,
    assignments = df_out,
    cluster_summary = cluster_summary,
    scaled_matrix = prep$scaled_matrix
  )
}
```

```r
kmeans_results <- run_kmeans_clustering(
  company_df = company_df,
  cluster_features = cluster_features,
  k = 4
)
```

```
## Warning: There was 1 warning in `summarise()`.
## i In argument: `across(all_of(cluster_features), mean, na.rm = TRUE)`.
## i In group 1: `cluster = 1`.
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
##   # Previously
##   across(a:b, mean, na.rm = TRUE)
##
##   # Now
##   across(a:b, \(x) mean(x, na.rm = TRUE))
```

```r
kmeans_results$cluster_summary
```

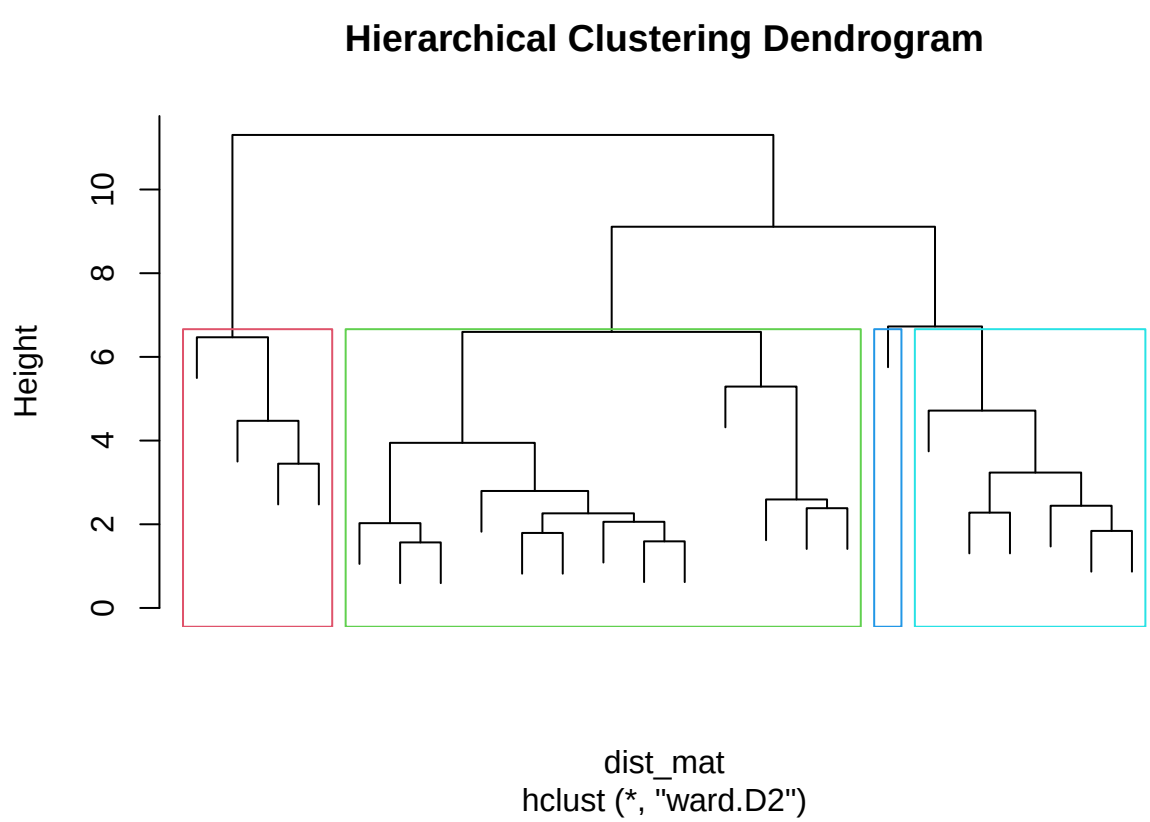
```
## # A tibble: 4 x 13
##   cluster n_companies rating_culture_values rating_work_life
##   <fct>         <int>                 <dbl>            <dbl>
## 1 1                 9                  3.32             3.56
## 2 2                 1                  4.25             4
## 3 3                12                  2.52             2.63
## 4 4                 2                  2                1.8
## # i 9 more variables: rating_career_opportunities <dbl>,
## #   rating_compensation_benefits <dbl>, rating_senior_leadership <dbl>,
## #   rating_diversity_inclusion <dbl>, helpfulness_score <dbl>,
## #   summary_wc <dbl>, pros_wc <dbl>, cons_wc <dbl>, advice_wc <dbl>
```

```r
head(kmeans_results$assignments)
```

```
## # A tibble: 6 x 15
##   company_name           sector n_reviews rating_culture_values rating_work_life
##   <chr>                  <chr>      <int>                 <dbl>            <dbl>
## 1 Accenture              Profe~        11                  3.73             3.45
## 2 Amazon                 Retail        14                  2.36             2.14
## 3 Anheuser-Busch InBev   Manuf~         5                  1.8              1
## 4 Bank of America        Finan~         5                  2.8              3.4
## 5 Commonwealth Bank of ~ Finan~         6                  3.67             3.67
## 6 GE                     Manuf~         5                  3                3.4
## # i 10 more variables: rating_career_opportunities <dbl>,
## #   rating_compensation_benefits <dbl>, rating_senior_leadership <dbl>,
## #   rating_diversity_inclusion <dbl>, helpfulness_score <dbl>,
## #   summary_wc <dbl>, pros_wc <dbl>, cons_wc <dbl>, advice_wc <dbl>,
## #   cluster <fct>
```

Hierarchical

```r
run_hclust <- function(company_df, cluster_features, k = 4) {
  prep <- prepare_company_clustering_data(company_df, cluster_features)

  dist_mat <- dist(prep$scaled_matrix)
  hc <- hclust(dist_mat, method = "ward.D2")

  cluster_assignments <- cutree(hc, k = k)

  df_out <- prep$data %>%
    mutate(cluster = factor(cluster_assignments))

  list(
    model = hc,
    assignments = df_out,
    scaled_matrix = prep$scaled_matrix
  )
}

hc_results <- run_hclust(company_df, cluster_features, k = 4)
plot(hc_results$model, labels = FALSE, main = "Hierarchical Clustering Dendrogram")
rect.hclust(hc_results$model, k = 4, border = 2:5)
```



Describe clusters by centroid profile

```r
describe_clusters <- function(cluster_assignments, cluster_features) {
  cluster_assignments %>%
```

```r
    group_by(cluster) %>%
    summarise(
      n_companies = n(),
      across(all_of(cluster_features), mean, na.rm = TRUE),
      .groups = "drop"
    )
}

describe_clusters(kmeans_results$assignments, cluster_features)
```

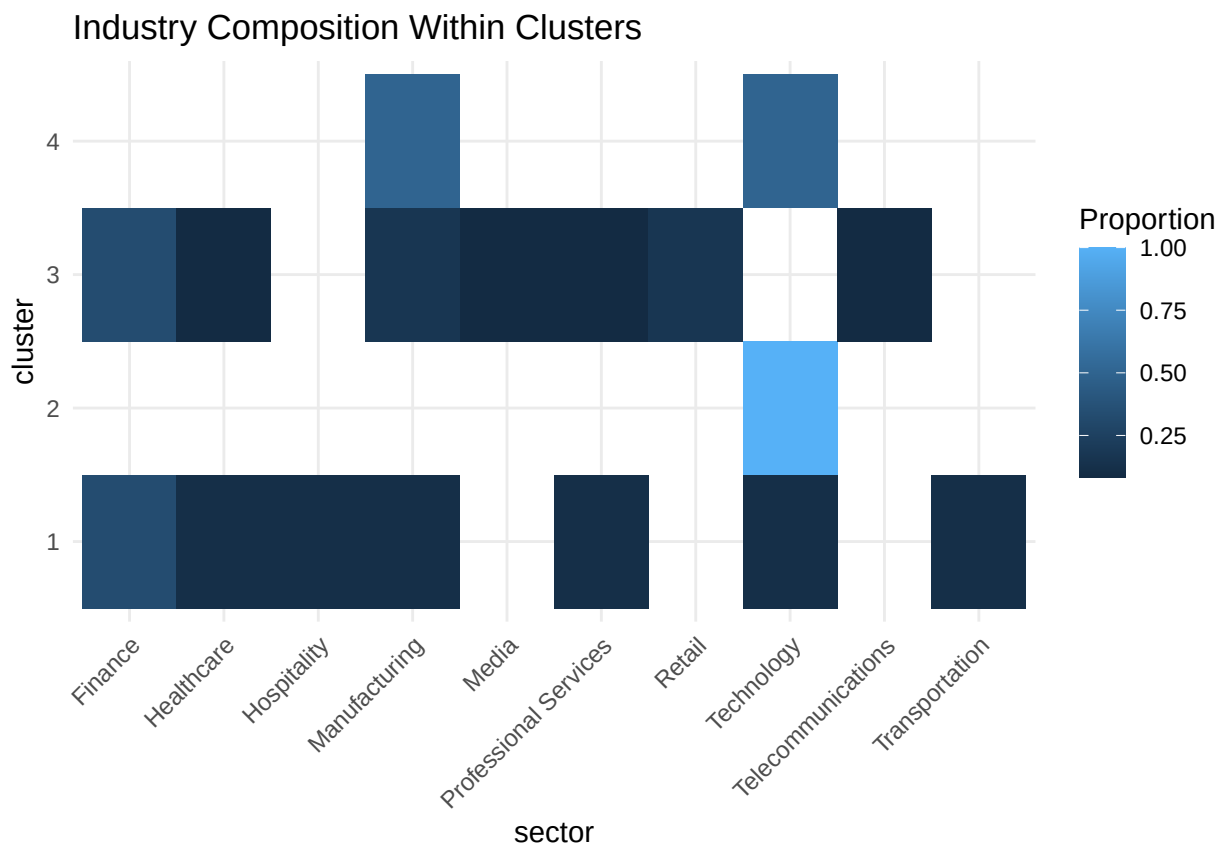
```
## # A tibble: 4 x 13
##   cluster n_companies rating_culture_values rating_work_life
##   <fct>         <int>                 <dbl>            <dbl>
## 1 1                 9                  3.32             3.56
## 2 2                 1                  4.25             4
## 3 3                12                  2.52             2.63
## 4 4                 2                  2                1.8
## # i 9 more variables: rating_career_opportunities <dbl>,
## #   rating_compensation_benefits <dbl>, rating_senior_leadership <dbl>,
## #   rating_diversity_inclusion <dbl>, helpfulness_score <dbl>,
## #   summary_wc <dbl>, pros_wc <dbl>, cons_wc <dbl>, advice_wc <dbl>
```

Compare clusters across industries

```r
cluster_sector_table <- kmeans_results$assignments %>%
  count(cluster, sector) %>%
  group_by(cluster) %>%
  mutate(cluster_pct = n / sum(n)) %>%
  ungroup()

cluster_sector_table
```

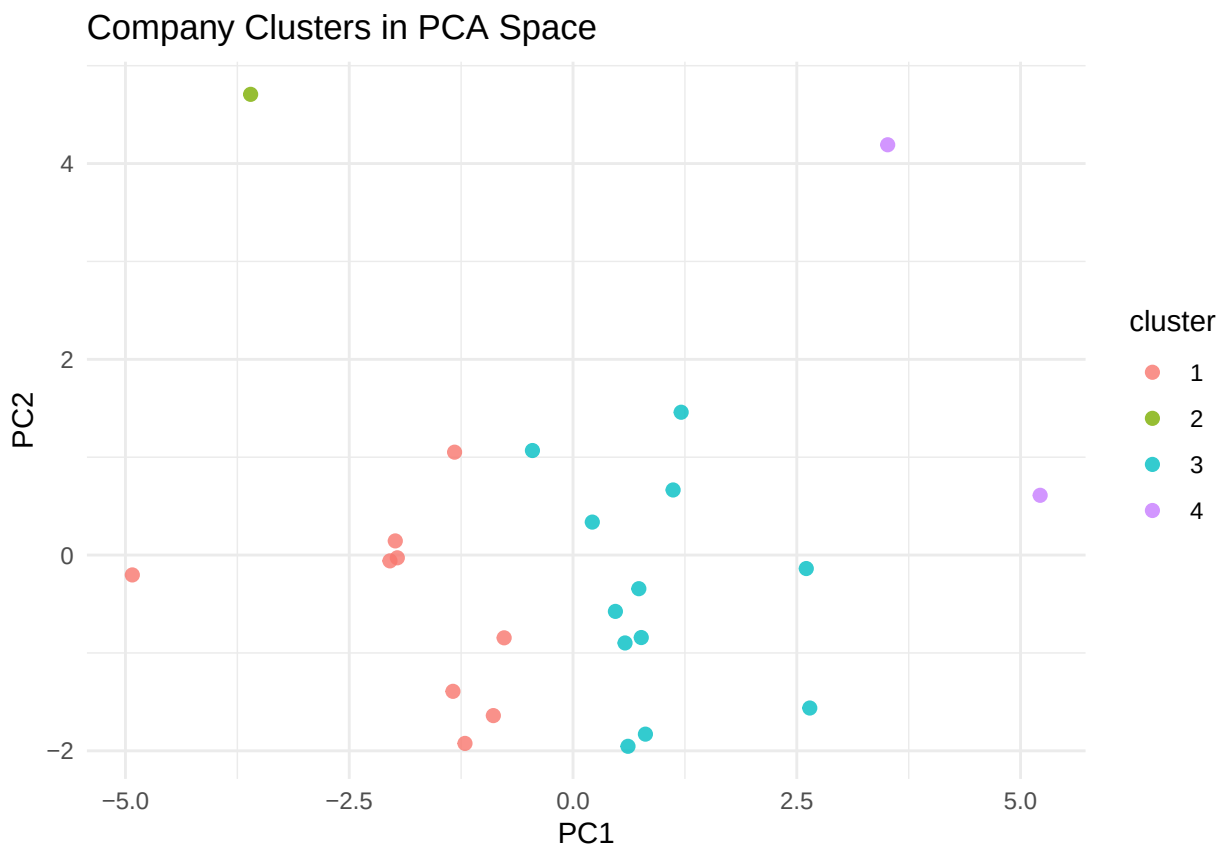
```
## # A tibble: 17 x 4
##    cluster sector                    n cluster_pct
##    <fct>   <chr>                 <int>       <dbl>
##  1 1       Finance                   3      0.333
##  2 1       Healthcare                1      0.111
##  3 1       Hospitality               1      0.111
##  4 1       Manufacturing             1      0.111
##  5 1       Professional Services     1      0.111
##  6 1       Technology                1      0.111
##  7 1       Transportation            1      0.111
##  8 2       Technology                1      1
##  9 3       Finance                   4      0.333
## 10 3       Healthcare                1      0.0833
## 11 3       Manufacturing             2      0.167
## 12 3       Media                     1      0.0833
## 13 3       Professional Services     1      0.0833
## 14 3       Retail                    2      0.167
## 15 3       Telecommunications        1      0.0833
## 16 4       Manufacturing             1      0.5
## 17 4       Technology                1      0.5
```

```r
ggplot(cluster_sector_table, aes(x = sector, y = cluster, fill = cluster_pct)) +
  geom_tile() +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(title = "Industry Composition Within Clusters", fill = "Proportion")
```



PCA

```r
plot_cluster_pca <- function(scaled_mat, cluster_labels) {
  pca <- prcomp(scaled_mat, center = FALSE, scale. = FALSE)

  plot_df <- data.frame(
    PC1 = pca$x[, 1],
    PC2 = pca$x[, 2],
    cluster = factor(cluster_labels)
  )

  ggplot(plot_df, aes(x = PC1, y = PC2, color = cluster)) +
    geom_point(size = 2, alpha = 0.8) +
    theme_minimal() +
    labs(title = "Company Clusters in PCA Space")
}
plot_cluster_pca(kmeans_results$scaled_matrix, kmeans_results$assignments$cluster)
```


Company Clusters in PCA Space
PC2
PC1
4
2
0
−2
−5.0
−2.5
0.0
2.5
5.0
cluster
1
2
3
4